

Astoria Open: System Architecture Document

Overview

The Astoria Open application is an advanced conversational AI platform designed to provide a unified, natural language interface to disparate data sources. It combines structured data from traditional SQL databases with unstructured text from document libraries, presenting a synthesized, panoramic view of the subject matter to the user. While currently configured for maritime history, it is built as a generic platform adaptable to any data domain.

1. User Features

From an end-user's perspective, the application provides the following key features:

Natural Language Querying: Users can ask questions in plain English, eliminating the need for technical knowledge of SQL or database schemas.

Unified Data Access: The system retrieves information from both a structured SQL database and an unstructured document library (vector database) simultaneously.

Synthesized Narrative Summaries: The application provides concise, AI-generated narratives that combine and summarize findings from all data sources.

Complex Analytical Capability: For complex or ambiguous questions, the system uses a powerful AI agent to explore the database, reason about the data, and provide a direct answer.

Transparent & Verifiable Results: Users can view the raw data tables and the exact SQL query that was generated by the AI, ensuring transparency and trust.

Query History: The interface maintains a session history of all questions asked, allowing users to easily revisit previous results.

2. Architecture Components

The application is built on a modern, multi-LLM architecture, with each component specializing in a specific task.

Frontend UI: A responsive Single-Page Application (SPA) built with React and Vite, running in the user's browser.

Backend Server: A high-performance FastAPI application running on a Gunicorn/Uvicorn server, which exposes the REST API.

Intelligent Router: A logic layer within the backend that analyzes incoming user queries and routes them to the appropriate tool.

Direct SQL Chain: The primary tool for simple, factual questions, using Google's Gemini API to convert a natural language question into an efficient SQL query.

SQL Agent: A powerful, autonomous agent powered by Google's Gemini API for complex, multi-step queries that require reasoning and database exploration.

Vector Database: A PostgreSQL database with the pgvector extension, hosted on Supabase, for fast semantic search on unstructured text.

Embeddings Model: A sentence-transformers model that converts queries and documents into numerical vectors.

Narrative Synthesizer: A fast Groq-powered Llama 3 model responsible for the final RAG step of synthesizing a cohesive, human-readable narrative from the structured (SQL) and unstructured (vector) results.

3. Detailed Data Flow

The application's data flow intelligently adapts based on the query's complexity, following one of two primary paths.

Simple Query Flow (e.g., "List 5 schooners")

The user submits the query through the React UI.

The FastAPI backend receives the request at the

/api/query endpoint.

The Intelligent Router identifies the query as simple.

The query is passed to the Direct SQL Chain.

The chain uses the Gemini API to generate an optimized SQL string.

The backend executes the SQL against the PostgreSQL Database to get structured

results.

Simultaneously, the query is converted into a vector to search the Vector Database for unstructured context.

Both results are passed to the Narrative Synthesizer.

The Groq LLM generates a final narrative summary.

The backend packages the response and sends it to the React UI.

Complex Query Flow (e.g., "Analyze the most common engine types")
The user submits the query through the React UI.

The FastAPI backend receives the request.

The Intelligent Router identifies the query as complex.

The query is passed to the SQL Agent.

The SQL Agent begins a reasoning process, using tools to explore the database tables and schema.

The Agent formulates and executes one or more SQL queries against the PostgreSQL Database.

After analyzing the results, the Agent formulates a final, text-based answer.

The backend packages the agent's answer and sends it to the React UI.

4. The Platform Architecture

This application was designed as a reusable platform for creating a natural language interface to any domain with both structured and unstructured data. The core architecture is data-agnostic. To adapt it to a new context, one only needs to connect a new SQL database, load new documents into the vector store, and update the system prompts to reflect the new domain's persona. The underlying FastAPI server, React UI, and data flow logic would remain unchanged, making it a powerful and flexible platform.

Ap